

Color Follow (No-PTZ)

1. Experiment Objective

The camera detects a target of a specified color, and the car body uses PD control for rotation tracking + distance following to keep the target centered in the frame and maintain a constant distance.

2. Experiment Principle

1. Key Terms

Term	Description
Visual Follow	Adds distance control (linear.x) on top of tracking, making the car rotate and move forward/backward simultaneously
Distance Follow	Estimates distance from the target's pixel size (color block radius), with PD control keeping the car at the desired distance
Body PD Control	Uses independent PD controllers for angle (angular.z) and distance (linear.x)
Lost-target Tolerance	When the target is temporarily lost, motion is allowed to continue for <code>lost_frame_threshold</code> (5) frames to avoid frequent stops
HSV Color Space	Describes colors in three dimensions — Hue (H), Saturation (S), Value (V) — more robust to lighting changes than RGB
Deadband Control	A horizontal non-responsive region at the center of the frame (<code>deadband_x_pixel</code> = 60 px) that prevents jitter when the target is near the center
QoS	<code>best_effort</code> (image input + cmd_vel downlink), <code>depth=1</code>

Difference Between Tracking and Following: Tracking only controls the body rotation to keep the target at the center of the frame (angular.z); following adds distance control (linear.x) on top of that, keeping the car at the desired distance from the target.

2. Technical Architecture

Shares the same node `color_tracker_node` as Color Tracking; distance following is enabled with `enable_distance_follow:=true`:

```
/camera/image_raw (BEST_EFFORT)
  → color_tracker_node (enable_distance_follow:=true)
    └─ Image processing: HSV binarization → largest contour → centroid + color
        block radius
      └─ Body control timer (10 Hz):
          └─ Pixel error → PD angular velocity → angular.z
              └─ color block radius error → PD linear velocity → linear.x → /cmd_vel
                  (BEST_EFFORT)
```

3. Core Topics Quick Reference

Firmware Downlink

Topic	Message Type	QoS	Field	Range	Direction
/cmd_vel	geometry_msgs/Twist	best_effort + keep_last(1)	linear.x	-0.12~0.12 m/s	+= forward (approach), -= backward (move away)
			angular.z	-0.60~0.60 rad/s	+= left turn, -= right turn

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ Version	✗
No-PTZ Version	☑
Required Peripherals	ESP32 camera (mounted on bracket)

2. Start Agent + Camera + Joint Model (Terminal 1)

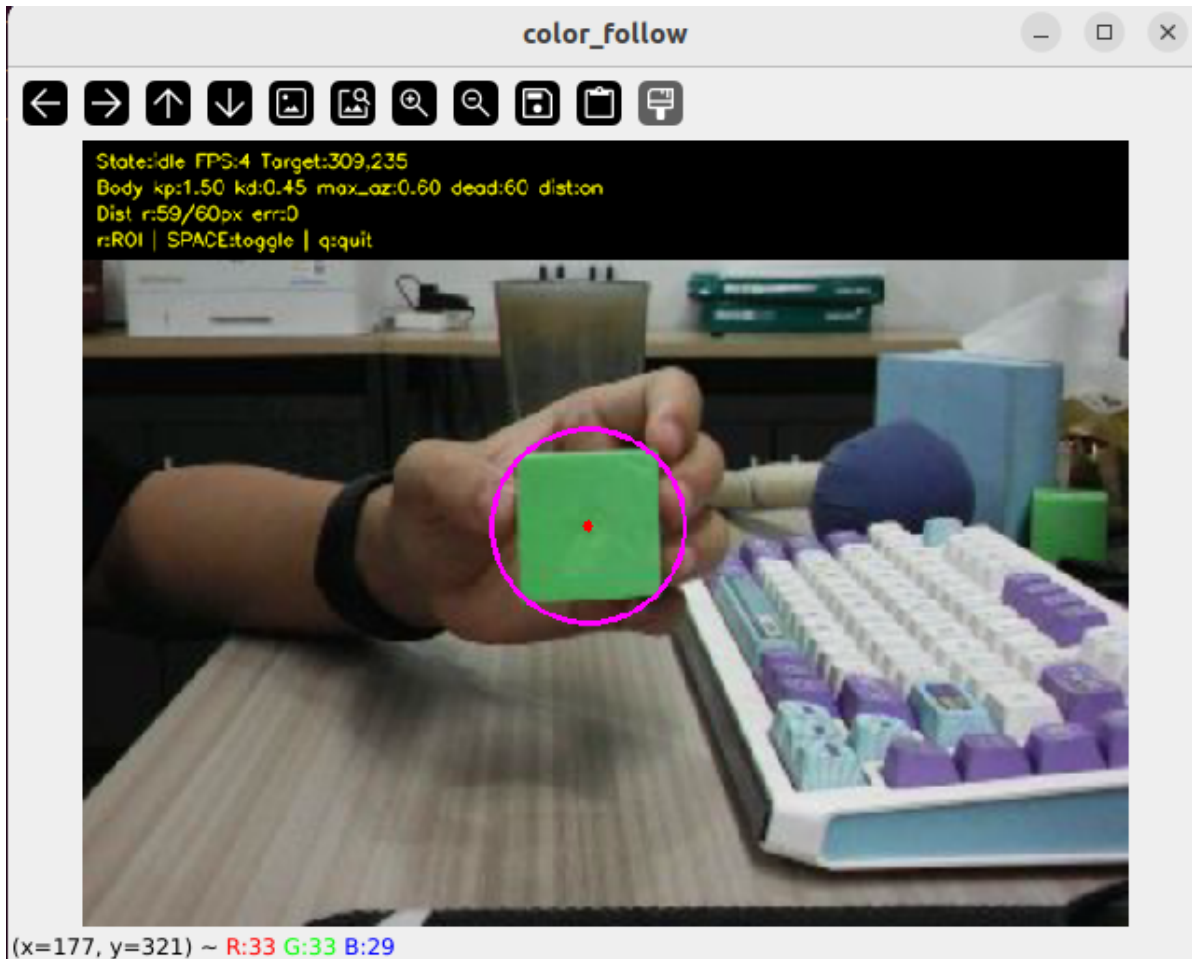
```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom: $ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment can2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment can3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.705741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start Color Follow (Terminal 2)

```
ros2 launch microros_v2_no_ptz_visual_control_pkg color_follow.launch.py
```

Follow mode is equivalent to `color_tracker.launch.py enable_distance_follow:=true`.



5. Operation Instructions

Key	Function
Space	Toggle follow/standby
q / ESC	Exit

Operation	Function
Left mouse button drag	Box-select the target color region and automatically compute the HSV thresholds

Distance Follow Behavior: The target approaches (color block radius increases) → move backward; the target moves away (radius decreases) → move forward; radius within tolerance → stay still.

4. Experiment Result

- Target approaches → the car automatically moves backward to keep the distance
- Target moves away → the car automatically moves forward to follow

- Distance within the desired range → the car stays still
- Target lost → the car automatically stops once the lost-target count exceeds the threshold

5. Code Explanation

Source code paths:

- Launch:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/color_follow.launch.py`
- Node source:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/color_tracker_node.py`

1. Core Node Explanation

`color_follow.launch.py` launches `color_tracker_node` (the same node shared with Color Tracking); the only difference is that `enable_distance_follow` defaults to `true`.

Distance Follow PD Logic (in the body control timer):

```
# Source file:
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/color_tracker_node.py

# Distance follow: color block radius error → PD linear velocity
if self._dist_follow and tr is not None:
    err_r = self._target_r_px - tr                # desired radius - actual radius
    if abs(err_r) > self._dist_tol_r:              # deadband check
        now = time.perf_counter()
        dt = now - self._prev_dt if self._prev_dt > 0 else 1.0 / self._body_rate
        dt = max(dt, 1e-3)
        d_err_r = (err_r - self._prev_derr) / dt
        self._prev_derr = err_r; self._prev_dt = now
        lx = self._dist_kp * err_r + self._dist_kd * d_err_r
        lx = clamp(lx, -self._max_lx, self._max_lx)
        cmd.linear.x = lx
```

Key Logic:

- `err_r > 0` (actual radius < desired = target far) → positive linear.x → move forward to approach
- `err_r < 0` (actual radius > desired = target close) → negative linear.x → move backward to retreat
- Radius error within `distance_tolerance_r` (12 px) → no adjustment (deadband)

2. Key Parameters

Parameter definition files:

- Launch:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/color_follow.launch.py`
- Node:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz`

Parameter	Type	Default	Description
<code>deadband_x_pixel</code>	float	60.0 px	Horizontal deadband
<code>angular_kp</code>	float	1.5	Angle PD proportional gain
<code>angular_kd</code>	float	0.45	Angle PD derivative gain
<code>max_angular_z</code>	float	0.6 rad/s	Maximum turning speed
<code>enable_distance_follow</code>	bool	true	Enable distance following
<code>target_radius</code>	float	60.0 px	Desired target color block radius
<code>distance_kp</code>	float	0.008	Distance PD proportional gain
<code>distance_kd</code>	float	0.003	Distance PD derivative gain
<code>distance_tolerance_r</code>	float	12.0 px	Distance tolerance deadband
<code>max_linear_x</code>	float	0.12 m/s	Maximum forward/backward speed
<code>lost_frame_threshold</code>	int	5	Lost-target tolerance frame count
<code>body_control_rate</code>	float	10.0 Hz	Body control rate

3. Node Description

Node	Package	Description
<code>color_tracker_node</code>	<code>microros_v2_no_ptz_visual_control_pkg</code>	Color tracking + following, dual timers + dual PD (angle + distance)

6. Common Issues

Issue	Cause	Solution
Sudden stop	Target loss triggers freezing	Keep the target in the frame and increase <code>lost_frame_threshold</code>
Poor distance following	Unreasonable distance PD parameters	Adjust <code>distance_kp</code> / <code>distance_kd</code> / <code>target_radius</code>
Car moves forward/backward frequently	Distance tolerance <code>distance_tolerance_r</code> too small	Increase the tolerance value

